

Doubly Linked Lists: Adding a Way Back

Programming and Data Structures — Week 4, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain precisely what a `prev` reference on each node buys that a singly linked list cannot offer; implement a doubly linked list that supports $O(1)$ tail insertion by maintaining an explicit `tail` reference; delete an arbitrary node in $O(1)$ time, given a direct reference to that node, without needing to search for its predecessor; and trace all four pointer updates a doubly linked insertion requires, in the correct order.

Outline

Today covers a train-carriage analogy for bidirectional linking; exactly what changes mechanically when a `prev` reference is added to each node; maintaining both `head` and `tail` references for $O(1)$ insertion at either end; the genuine advantage this unlocks, $O(1)$ deletion given a direct node reference, without a predecessor search; traversal in both directions; browser history as a worked application; the MTech-track connection to how Python's own `collections.deque` is implemented internally; an in-class quiz; and a summary.

The train-carriage analogy

Picture a train where each carriage is coupled only to the carriage ahead of it — this is Lecture 1's singly linked list: standing in any carriage, you can walk toward the front, but to reach the very back you must walk the entire length of the train, carriage by carriage, with no way to check behind you directly. A doubly linked list adds a second coupling to each carriage, connecting it to the carriage behind as well as the one ahead. From any carriage, you can now walk in either direction — forward toward the engine, or backward toward the caboose — without needing to have kept track of where you came from.

Adding prev: the node changes

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None    # same as Lecture 1
        self.prev = None    # NEW: reference to the previous node
```

The only structural change from Lecture 1's Node is the addition of a single new field, `prev`, holding a reference to the previous node in the chain. Every node in the middle of the list now has two directly nameable neighbors — the one ahead via `next`, and the one behind via `prev`. The list's boundaries are still clearly marked: the first node's `prev` is `None`, and the last node's `next` is `None`, exactly mirroring how the singly linked list marked its single end.

The DoublyLinkedList wrapper: two references now

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None    # first node
        self.tail = None    # NEW: last node, tracked explicitly
        self._size = 0

    def __len__(self):
        return self._size
```

Lecture 1's `LinkedList` tracked only `head`, which is why `push_back` had to traverse the entire list to find the current last node — an $O(n)$ cost. This wrapper class tracks `tail` explicitly as a second reference, updated on every insertion or deletion at the end, so the last node is reachable in $O(1)$ directly, with no traversal required at all. This single addition is what turns tail-insertion from $O(n)$ into $O(1)$.

`push_back`: now genuinely $O(1)$

```
def push_back(self, value):
    new_node = Node(value)
    if self.tail is None:    # empty list
        self.head = self.tail = new_node
```

```

else:
    new_node.prev = self.tail      # 1. new node's prev points at old tail
    self.tail.next = new_node     # 2. old tail's next points at new node
    self.tail = new_node         # 3. tail reference itself moves
    self._size += 1

```

With `tail` tracked directly, `push_back` becomes a small, fixed number of reference updates — no traversal at all. The new node's `prev` is set to point at the current `tail`; the current `tail`'s `next` is set to point at the new node; and finally the `tail` reference itself is updated to the new node. All three steps are $O(1)$, regardless of how long the list already is — a direct improvement over Lecture 1's $O(n)$ `push_back`, purchased entirely by maintaining one additional reference.

Tracing `push_back`, step by step

Before: `head` → [10] [20] [30] ← `tail`

`push_back(40):`

`new_node` = [40]

Step 1: `new_node.prev` = `tail` → [40].`prev` points at [30]

Step 2: `tail.next` = `new_node` → [30].`next` points at [40]

Step 3: `tail` = `new_node` → `tail` now IS [40]

After: `head` → [10] [20] [30] [40] ← `tail`

Tracing `push_back(40)` on a three-node list: the new node's `prev` is linked backward to the current `tail` (30) first, then the current `tail`'s `next` is linked forward to the new node, and only then does the `tail` reference itself move to point at the new node. Getting this order wrong — moving `tail` before linking `30.next`, for instance — would lose the reference needed to complete the link, the same ordering discipline Lecture 1 emphasized for head-insertion.

The real win: $O(1)$ deletion given a node reference

```

def remove_node(self, node):
    if node.prev is not None:
        node.prev.next = node.next
    else:
        self.head = node.next      # node was the head
    if node.next is not None:
        node.next.prev = node.prev

```

```

else:
    self.tail = node.prev          # node was the tail
    self._size -= 1

```

This is the central advantage a doubly linked list provides over a singly linked one. Given a direct reference to the node to be deleted — not its value, the actual node object — no search for its predecessor is required at all, because `node.prev` already names it directly. The deletion simply redirects the neighbor on each side to skip over `node`, handling the head and tail boundary cases separately. This is genuinely $O(1)$, a real structural improvement over Lecture 1’s singly linked list, which required an $O(n)$ search specifically to find the node *before* the target.

Tracing `remove_node`, step by step

Before: head → [10] [20] [30] ← tail (removing the node holding 20)

```

node = the [20] node (already have this reference - no search)
node.prev is [10] (not None) → [10].next = node.next = [30]
node.next is [30] (not None) → [30].prev = node.prev = [10]

```

After: head → [10] [30] ← tail

Tracing the removal of the middle node holding 20, given a direct reference to it: since `node.prev` is the node holding 10 (not `None`), that node’s `next` is redirected to skip over 20, pointing straight at 30. Since `node.next` is the node holding 30 (not `None`), that node’s `prev` is redirected the same way, pointing straight back at 10. Both neighbors are updated using only references already available on `node` itself — no traversal of the list was needed at any point.

Why this matters: it is not just “faster deletion”

It is worth being precise about why this matters: a singly linked list cannot achieve $O(1)$ deletion given only a node reference, no matter how cleverly it is implemented, because it structurally lacks any way to reach the predecessor without a search — the `prev` field simply does not exist. Any application that frequently needs to delete “the node I am currently holding onto” benefits directly from this doubly linked structure. This is precisely the mechanism behind an efficient Least Recently Used (LRU) cache, a structure this course returns to in Week 8 when hash tables are combined with exactly this kind of linked list.

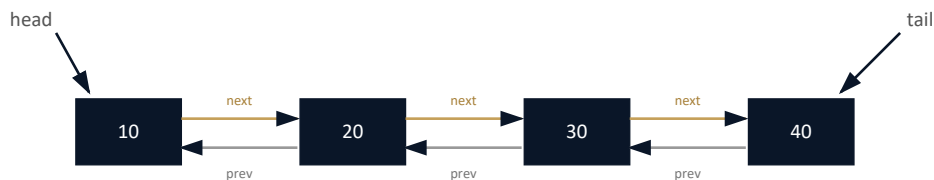
Traversal in both directions

```
def to_list_forward(self):
    result, current = [], self.head
    while current is not None:
        result.append(current.value)
        current = current.next
    return result

def to_list_backward(self):
    result, current = [], self.tail
    while current is not None:
        result.append(current.value)
        current = current.prev
    return result
```

Traversal can now proceed in either direction: forward from head following `.next`, exactly as in a singly linked list, or backward from tail following `.prev`, a capability a singly linked list simply does not have. Both traversals cost $O(n)$, same as before — the improvement this lecture introduces is not about traversal speed, but about which operations become possible or cheap at all.

The picture: two arrows between every pair of nodes



Every consecutive pair is now linked in both directions — walk forward via `next`, or backward via `prev`, from any node.

Drawing both the forward and backward arrows between every consecutive pair of nodes makes the structural difference from Lecture 1 immediately visible — every link in a singly linked list becomes two links here, one in each direction.

Application: browser history, forward and back

```

history = DoublyLinkedList()
history.push_back("home.html")
history.push_back("courses.html")
history.push_back("z2005/index.html")
current = history.tail           # currently viewing z2005/index.html

def go_back(current):
    return current.prev if current.prev else current

def go_forward(current):
    return current.next if current.next else current

```

Browser history is a natural application of a doubly linked list: each visited page is a node, `next` represents “forward,” and `prev` represents “back.” A user currently viewing `z2005/index.html` (the tail) can go back to `courses.html` by following `.prev`, and from there, forward again to `z2005/index.html` by following `.next` — both directions are genuinely symmetric operations here, which is exactly the browsing behavior a real back/forward button pair needs to support.

MTech addition — this is how `collections.deque` really works

Week 2 stated that `collections.deque` is internally a doubly linked list of fixed-size blocks, without proving the connection at the time. This lecture now supplies the missing piece: deque links blocks — not individual elements — using exactly the `prev/next` pointer structure just built, with each block internally holding a small fixed-size array of several elements. This hybrid design achieves $O(1)$ operations at both ends, inherited from the doubly linked block chain, while retaining better memory locality than linking every single element individually — a real engineering trade-off between the array’s cache-friendliness and the linked list’s $O(1)$ end operations.

Cost summary, three structures now compared

Extending Lecture 1’s comparison table to include the doubly linked list makes the progression across this week clear: doubly linking matches the array’s $O(1)$ back-insertion (though via a different mechanism, an explicit `tail` reference rather than amortized doubling) and adds a capability neither the array nor the singly linked list has — $O(1)$ deletion given a direct node reference. What doubly linking does *not* fix is index access, which remains $O(n)$ for both linked variants, since reaching an arbitrary position still requires traversal from one end.

Exercise

As an exercise, implement `push_front` for `DoublyLinkedList`, correctly updating both the head reference and the appropriate `prev/next` pointers on the affected nodes. Then implement `remove_value(value)`, which first searches for the node holding that value (necessarily $O(n)$, since the value, not a node reference, is what is given) and then calls `remove_node` on it ($O(1)$, as shown in this lecture). Trace `remove_value(20)` on the list `[10] [20] [30]`, explicitly identifying which part of the operation is $O(n)$ and which part is $O(1)$ — this distinction is a common point of confusion worth clarifying by hand.

The memory cost of the second reference

It is worth quantifying the cost of this improvement, not just its benefit. Every node now stores one additional reference, `prev`, which on a typical 64-bit system costs about 8 bytes per node. For a list holding a million nodes, this amounts to roughly 8 megabytes of additional memory used purely for `prev` fields, memory a singly linked list would not need to spend at all. This is the concrete, quantifiable price of gaining $O(1)$ deletion given a node reference — a real trade-off, not a free upgrade, though one that is usually well worth paying in practice given how often that capability matters.

Quiz — cost, given what you're handed

Two related questions, before checking the answer: given a direct reference to a node in the middle of a doubly linked list (not its value, the actual node object), what is the cost of deleting it? Separately, given only the value stored in that node, with no direct reference, what is the cost of deleting it?

Quiz — answer

Given a direct reference to the node, deletion is $O(1)$, using `remove_node` exactly as traced in this lecture — no search is needed because both neighbors are reachable directly through `prev` and `next`. Given only a value, deletion is $O(n)$ overall, because a search is required first to locate the node holding that value, even though the actual removal step, once the node is found, is still $O(1)$. The distinction is entirely about what information is already available at the call site, not about any change in the list's internal structure — this is a subtlety worth being precise about, since both cases use the identical `remove_node` machinery underneath.

Summary

Adding a `prev` reference to each node allows any node to be reached from either direction, not just forward as in Lecture 1's singly linked list. Tracking both `head` and `tail` explicitly turns insertion at either end into a genuine $O(1)$ operation. The structural advantage this lecture introduces is $O(1)$ deletion given a direct reference to the node — an operation a singly linked list cannot achieve at all, regardless of implementation cleverness, because it lacks the `prev` field needed to reach the predecessor without a search. Real systems rely on exactly this property: browser history navigation, the LRU cache structure returning in Week 8, and the block-chain implementation of Python's own `collections.deque`. Next lecture examines circular linked lists, where the chain loops back on itself, and the classic Josephus problem this structure solves naturally.